

Competitive Programming

Section 38 Editorial: Static Queries and Binary Lifting

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	4
2	A. Static Range Minimum	5
2.1	Restatement	5
2.2	Algorithm	5
2.3	Correctness proof	5
2.4	Complexity	5
2.5	Full C++ solution	5
2.6	Full Python solution	6
3	B. Static Range GCD	8
3.1	Restatement	8
3.2	Algorithm	8
3.3	Correctness proof	8
3.4	Complexity	8
3.5	Full C++ solution	8
3.6	Full Python solution	9
4	C. Kth Ancestor	11
4.1	Restatement	11
4.2	Algorithm	11
4.3	Correctness proof	11
4.4	Complexity	11
4.5	Full C++ solution	11
4.6	Full Python solution	12
5	D. Ancestor Minimum	14
5.1	Restatement	14
5.2	Constraints and traps	14
5.3	Observations	14
5.4	Sample walk-through	15
5.5	Algorithm	15
5.6	Correctness proof	15
5.7	Complexity	16
5.8	C++ implementation notes	16
5.9	Python implementation notes	16
5.10	Full C++ solution	16
5.11	Full Python solution	17
5.12	Common wrong answers	18
5.13	Tests that matter	19
6	Testing discussion	20
7	Problem-specific upsolve notes	21
7.1	A. Static Range Minimum	21

7.2	B. Static Range GCD	21
7.3	C. Kth Ancestor	21
8	Stress-testing workflow	22

1 Editorial overview

Problem	Main idea
A. Static Range Minimum	See full algorithm and code below.
B. Static Range GCD	See full algorithm and code below.
C. Kth Ancestor	See full algorithm and code below.
D. Ancestor Minimum	Binary-lift an ancestor path while merging minimum labels.

2 A. Static Range Minimum

2.1 Restatement

Given an immutable array, answer minimum queries on subarrays.

2.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

2.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

2.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

2.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
```

```

if (!(cin >> n >> q)) return 0;
vector<int> lg(n + 1);
for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
int K = lg[n] + 1;
vector<vector<long long>> st(K, vector<long long>(n));
for (int i = 0; i < n; i++) cin >> st[0][i];
for (int k = 1; k < K; k++) {
    for (int i = 0; i + (1 << k) <= n; i++) {
        st[k][i] = min(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
    }
}
while (q--) {
    int l, r;
    cin >> l >> r;
    --l; --r;
    int k = lg[r - l + 1];
    cout << min(st[k][l], st[k][r - (1 << k) + 1]) << '\n';
}
return 0;
}

```

2.6 Full Python solution

```

import sys, math

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    a = data[2:2 + n]
    log = [0] * (n + 1)
    for i in range(2, n + 1):
        log[i] = log[i // 2] + 1
    st = [a]
    k = 1
    while (1 << k) <= n:
        prev = st[-1]
        length = 1 << (k - 1)
        st.append([min(prev[i], prev[i + length]) for i in range(n - (1 << k) + 1)])
        k += 1
    idx = 2 + n
    out = []
    for _ in range(q):
        l, r = data[idx] - 1, data[idx + 1] - 1
        idx += 2
        k = log[r - l + 1]
        out.append(str(min(st[k][l], st[k][r - (1 << k) + 1])))

```

```
    print("\n".join(out))  
  
if __name__ == "__main__":  
    main()
```

3 B. Static Range GCD

3.1 Restatement

Given an immutable positive array, answer gcd queries on subarrays.

3.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

3.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

3.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

3.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
```

```

if (!(cin >> n >> q)) return 0;
vector<int> lg(n + 1);
for (int i = 2; i <= n; i++) lg[i] = lg[i / 2] + 1;
int K = lg[n] + 1;
vector<vector<long long>> st(K, vector<long long>(n));
for (int i = 0; i < n; i++) cin >> st[0][i];
for (int k = 1; k < K; k++) {
    for (int i = 0; i + (1 << k) <= n; i++) {
        st[k][i] = gcd(st[k - 1][i], st[k - 1][i + (1 << (k - 1))]);
    }
}
while (q--) {
    int l, r;
    cin >> l >> r;
    --l; --r;
    int k = lg[r - l + 1];
    cout << gcd(st[k][l], st[k][r - (1 << k) + 1]) << '\n';
}
return 0;
}

```

3.6 Full Python solution

```

import math
import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    a = data[2:2 + n]
    log = [0] * (n + 1)
    for i in range(2, n + 1):
        log[i] = log[i // 2] + 1
    st = [a]
    k = 1
    while (1 << k) <= n:
        prev = st[-1]
        length = 1 << (k - 1)
        st.append([math.gcd(prev[i], prev[i + length]) for i in range(n - (1 << k) + 1)])
        k += 1
    idx = 2 + n
    out = []
    for _ in range(q):
        l, r = data[idx] - 1, data[idx + 1] - 1
        idx += 2
        k = log[r - l + 1]

```

```
        out.append(str(math.gcd(st[k][l], st[k][r - (1 << k) + 1])))
    print("\n".join(out))

if __name__ == "__main__":
    main()
```

4 C. Kth Ancestor

4.1 Restatement

In a rooted tree with root 1, answer queries asking for the k -th ancestor of a node, or -1 if it does not exist.

4.2 Algorithm

Identify the operation that must be fast, store exactly the aggregate information needed for that operation, and update the affected logarithmic frontier. The lesson for this section explains the invariant in detail; the implementation below is a direct transcription of that invariant.

For this problem, every query is processed in the order given except offline queries explicitly marked as offline. The answer is emitted immediately when the operation asks for one.

4.3 Correctness proof

The maintained structure stores the exact summary promised by the lesson: prefix blocks for Fenwick trees, segment aggregates for segment trees, power-of-two jumps for static structures, or the active frontier for heap/Mo-style processing. Each update changes precisely the summaries whose represented set contains the changed element or interval. Each query decomposes the requested object into summaries that are disjoint, idempotently overlapping, or maintained as the current active window. The merge formula for those summaries is exactly the mathematical definition of the requested answer, so every printed value is correct. $\square$

4.4 Complexity

The optimized operations are logarithmic unless the lesson says the structure is static or offline. Memory is linear or $O(n \log n)$ for sparse-table/binary-lifting preprocessing.

4.5 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

int n, q;
if (!(cin >> n >> q)) return 0;
int LOG = 1;
while ((1 << LOG) <= n + 5) LOG++;
vector<vector<int>> up(LOG, vector<int>(n + 1, 0));
for (int v = 2; v <= n; v++) cin >> up[0][v];
for (int k = 1; k < LOG; k++) {
    for (int v = 1; v <= n; v++) up[k][v] = up[k - 1][up[k - 1][v]];
}
while (q--) {
    int v;
    long long dist;
    cin >> v >> dist;
    for (int k = 0; k < LOG && v; k++) {
        if (dist & (1LL << k)) v = up[k][v];
    }
    cout << (v ? v : -1) << '\n';
}
return 0;
}

```

4.6 Full Python solution

```

import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    LOG = max(1, (n + 5).bit_length())
    up = [[0] * (n + 1) for _ in range(LOG)]
    idx = 2
    for v in range(2, n + 1):
        up[0][v] = data[idx]
        idx += 1
    for k in range(1, LOG):
        for v in range(1, n + 1):
            up[k][v] = up[k - 1][up[k - 1][v]]
    out = []
    for _ in range(q):
        v, dist = data[idx], data[idx + 1]
        idx += 2
        bit = 0
        while dist and v:
            if dist & 1:
                v = up[bit][v] if bit < LOG else 0
            dist >>= 1
            bit += 1
    for v in out:
        print(v)

```

```
        out.append(str(v if v else -1))
    print("\n".join(out))

if __name__ == "__main__":
    main()
```

5 D. Ancestor Minimum

5.1 Restatement

For each (v, k) , inspect v , its parent, and every ancestor through the k -th ancestor. Print their smallest label, or -1 if that ancestor does not exist. This blind exercise looks like k th-ancestor queries, but it asks for an aggregate over the nodes crossed by the jumps.

5.2 Constraints and traps

$n, q \leq 200,000$, so climbing one edge at a time is quadratic on a chain. Labels can be negative, so zero is not a valid missing-value sentinel. The path includes v itself: a query with $k=0$ returns $\text{label}[v]$. A one-edge aggregate must include both v and its parent, not just the start node.

5.3 Observations

Keep the normal binary-lifting table:

```
up[t][v] = node exactly  $2^t$  parent edges above  $v$ , or 0 if absent.
```

Also store an inclusive path aggregate whenever that jump exists:

```
best[t][v] = minimum label from  $v$  through  $\text{up}[t][v]$ , inclusive.
```

For one edge:

```
best[0][v] =  $\min(\text{label}[v], \text{label}[\text{parent}[v]])$ .
```

For a larger jump, let $\text{mid} = \text{up}[t-1][v]$. Two adjacent half-jumps form the whole path:

```
up[t][v] = up[t-1][mid]  
best[t][v] =  $\min(\text{best}[t-1][v], \text{best}[t-1][\text{mid}])$ 
```

To answer (v, k) , start $\text{answer} = \text{label}[v]$. Decompose k into set bits. For each selected bit t , reject if $\text{up}[t][v]$ is zero; otherwise merge $\text{best}[t][v]$ and move v to $\text{up}[t][v]$. Consecutive segments may repeat an endpoint, but minimum is idempotent, so that overlap changes nothing.

5.4 Sample walk-through

For (5,2), the path is 5 → 2 → 1 with labels 6,5,8. Bit one is set, so take a two-edge jump:

```
best[1][5] = min(best[0][5], best[0][2])
            = min(min(6,5), min(5,8)) = 5.
```

The answer is 5. For (4,3), the third parent edge would leave the root, so the answer is -1, not the minimum on the shorter existing prefix.

5.5 Algorithm

```
build up[0][v] = parent[v]
build best[0][v] = min(label[v], label[parent[v]])

for t = 1..LOG-1:
    mid = up[t-1][v]
    up[t][v] = up[t-1][mid]
    best[t][v] = min(best[t-1][v], best[t-1][mid])

for query (v,k):
    answer = label[v]
    for every set bit t of k:
        if up[t][v] is zero: print -1
        otherwise merge best[t][v], then v = up[t][v]
    print answer if all selected jumps existed
```

Use a large positive INF for aggregate entries involving sentinel node zero. The query checks the jump before using its aggregate, so INF never becomes a real answer.

5.6 Correctness proof

Lemma 1. If $up[t][v]$ exists, it is the 2^t -th ancestor of v , and $best[t][v]$ is the minimum on the inclusive path between them.

Proof. At $t=0$, the table stores the parent and the minimum of its two endpoint labels. For larger t , the recurrence joins the path from v to mid and the adjacent path from mid to $up[t-1][mid]$. By induction both half aggregates are correct; their minimum is exactly the aggregate on the combined 2^t -edge path. $\square$

Lemma 2. After processing selected query bits totaling d , the current node is the d -th ancestor of the original node and **answer** is the minimum on the path to it.

Proof. It holds initially for $d=0$. Lemma 1 moves the node by each selected power-of-two distance and merges exactly the labels crossed. Induction over the bit loop proves the claim. $\square$

Theorem. The algorithm prints -1 exactly for impossible queries and the requested path minimum for all other queries.

Proof. A missing selected jump proves that the requested ancestor is absent. Otherwise the selected powers sum to k , and Lemma 2 gives the minimum from the starting node through its k -th ancestor. $\square$

5.7 Complexity

With $L = \lceil \log_2(n+1) \rceil$:

```
Preprocessing:  $O(n L)$  time and memory  
Each query:    $O(L)$  time
```

5.8 C++ implementation notes

Use zero as the missing-parent sentinel and initialize `minimum[*][0]` to `INF`. Keep labels and table aggregates in `long long`. Before merging a selected jump, test `up[bit][node] != 0`; this distinguishes an impossible query from a valid path with a negative answer.

5.9 Python implementation notes

Use `10**30` as `INF` and an output list for all query answers. The loop that consumes k must advance its bit counter for both zero and one bits.

5.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, q;
    if (!(cin >> n >> q)) return 0;
    int log = 1;
    while ((1 << log) <= n) ++log;

    const long long INF = (1LL << 62);
    vector<long long> label(n + 1);
    for (int node = 1; node <= n; ++node) cin >> label[node];

    vector<vector<int>> up(log, vector<int>(n + 1, 0));
    vector<vector<long long>> minimum(log, vector<long long>(n + 1, INF));
    for (int node = 2; node <= n; ++node) cin >> up[0][node];
    for (int node = 1; node <= n; ++node) {
        minimum[0][node] = min(label[node], up[0][node] ? label[up[0][node]] : INF);
    }
}
```

```

    }

    for (int bit = 1; bit < log; ++bit) {
        for (int node = 1; node <= n; ++node) {
            int middle = up[bit - 1][node];
            up[bit][node] = up[bit - 1][middle];
            minimum[bit][node] = min(minimum[bit - 1][node], minimum[bit - 1][middle]);
        }
    }

    while (q--) {
        int node;
        long long distance;
        cin >> node >> distance;
        long long answer = label[node];
        bool exists = true;
        for (int bit = 0; distance; ++bit, distance >>= 1LL) {
            if ((distance & 1LL) == 0) continue;
            if (bit >= log || up[bit][node] == 0) {
                exists = false;
                break;
            }
            answer = min(answer, minimum[bit][node]);
            node = up[bit][node];
        }
        cout << (exists ? answer : -1) << '\n';
    }
    return 0;
}

```

5.11 Full Python solution

```

import sys

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    n, q = data[0], data[1]
    labels = [0] + data[2:2 + n]
    index = 2 + n
    log = max(1, n.bit_length())
    infinity = 10**30
    up = [[0] * (n + 1) for _ in range(log)]
    minimum = [[infinity] * (n + 1) for _ in range(log)]
    for node in range(2, n + 1):
        up[0][node] = data[index]

```

```

        index += 1
    for node in range(1, n + 1):
        minimum[0][node] = min(labels[node], labels[up[0][node]] if up[0][node] else infinity)

    for bit in range(1, log):
        previous_up = up[bit - 1]
        previous_minimum = minimum[bit - 1]
        for node in range(1, n + 1):
            middle = previous_up[node]
            up[bit][node] = previous_up[middle]
            minimum[bit][node] = min(previous_minimum[node], previous_minimum[middle])

    output: list[str] = []
    for _ in range(q):
        node = data[index]
        distance = data[index + 1]
        index += 2
        answer = labels[node]
        exists = True
        bit = 0
        while distance:
            if distance & 1:
                if bit >= log or up[bit][node] == 0:
                    exists = False
                    break
                answer = min(answer, minimum[bit][node])
                node = up[bit][node]
                distance >>= 1
                bit += 1
            output.append(str(answer if exists else -1))
    sys.stdout.write("\n".join(output) + "\n")

if __name__ == "__main__":
    main()

```

5.12 Common wrong answers

Storing only label[v] in best[0][v] and omitting the parent endpoint.
 Returning a partial-path minimum after discovering a missing ancestor.
 Using zero as the missing aggregate despite negative labels.
 Forgetting that k=0 includes the starting node.
 Choosing too few lifting rows for a deepest valid path.

5.13 Tests that matter

A one-node tree with $k=0$ and positive k .

Negative labels along a path.

A query that reaches exactly the root and one edge beyond it.

A long chain with 200,000 queries, to reject literal climbing.

6 Testing discussion

The fixed samples cover visible behavior. The random tests generate many small operation sequences and compute expected outputs from the reference Python implementation, then the same expected files are used against both the C++ and Python submissions.

7 Problem-specific upsolve notes

7.1 A. Static Range Minimum

Constraint implication and trap. The array never changes, so pay $O(n \log n)$ once for $O(1)$ queries. Do not use this overlapping two-block query trick for sum: it is valid here only because min is idempotent.

For $[5, 2, 7, 1, 3, 4]$, query $[2, 5]$ has length 4 and reads one 2^2 block; query $[2, 6]$ uses blocks $[2, 5]$ and $[3, 6]$, whose overlap cannot alter a minimum. Pseudocode is: build $st[k][i] = \min(st[k-1][i], st[k-1][i + 2^{k-1}])$; for $[l, r]$, set $k = \lg[r-l+1]$ and take the min of the two length- 2^k blocks. Induction proves each table cell; the two blocks cover the query, proving the answer. Preprocessing/memory $O(n \log n)$, query $O(1)$. Test singleton and non-power-of-two ranges; convert input indices exactly once.

7.2 B. Static Range GCD

Use the identical table shape with gcd as merge. For $[12, 18, 6, 10, 15]$, query $[2, 5]$ has gcd 1. Gcd is associative and idempotent, so the same overlapping-cover proof applies. Complexity is again $O(n \log n)$ build and memory, $O(1)$ per query. C++ uses `std::gcd`; Python uses `math.gcd`. Test a length-one range, equal values, and a range whose answer becomes one early.

7.3 C. Kth Ancestor

Observation and trace. Let $up[k][v]$ be the exact 2^k -th ancestor, with zero meaning absent. Build by composing two half jumps. To lift 4 by three in the sample tree, take bit 0 to reach 2, then bit 1 to reach zero, so output -1; for distance zero output 4 unchanged.

```
build up[0][v]=parent[v]
up[k][v]=up[k-1][up[k-1][v]]
for each set bit k of distance: v=up[k][v]
answer -1 if v=0 else v
```

Induction on k proves the table's jump meaning; binary expansion uses exactly the requested number of parent edges, proving each query. Build/memory are $O(n \log n)$ and a query is $O(\log n)$. Choose LOG for the largest possible query distance, not just an arbitrary constant. Test root, zero distance, one-step, and a distance above depth.

8 Stress-testing workflow

Compare sparse-table answers to direct scans on small arrays. For ancestor queries, climb the parent array literally. Run `CP_TARGET=solution python3 sections/38_static_queries_binary_lifting/check.py`. Carry forward: overlap needs idempotence; binary lifting is the same power-of-two decomposition applied to functional-graph/tree jumps.